VibeCoding Lab 4 — Whack-a-Mole Game

AI-Assisted Coding, Debugging and Testing Report

SRN: PES2UG24CS610

Name: Yashas Sadananda

Project: Whack-a-Mole Game

Repository: https://github.com/yxshas565/48_whack-a-mole

Purpose: Fix the existing gameplay bug and implement the features described in the project README using AI coding assistance.

1. Bug Identification and Fix

Problem before the fix: In the original game, a mouse click was checked against every hole's hitbox. Because the hitboxes were set to 150 pixels and could overlap, a click near the boundary between neighboring holes could be detected by more than one hole. If more than one matching hole had an active mole, the same physical click could trigger multiple successful whacks and increase the score by more than one point.

How this appeared in the original behavior: The player could click once but see the score increase unexpectedly by two points when overlapping hitboxes matched multiple holes. This is the bug the before-fix recording is intended to represent; the short video may not capture the issue on every attempt because it depends on the mole positions and click location.

Original faulty approach:

```
for hole in self.holes:
    if hole.rect.collidepoint(pos):
        if hole.whack():
            self.score += 1
```

Fix applied: We reduced the hole hitbox size from 150 pixels to 64 pixels and changed click handling so that at most one eligible active hole is selected for each click. A successful click awards one point only, while clicking empty space does not increase the score.

After the fix: As shown in the after-fix video, the updated game uses the corrected click handling so a single click can award at most one point, preventing the original double-scoring behavior.

2. Implementation Process and Features

Step 1: Inspect the project. We identified the main files involved: `main.py`, `game/game_engine.py`, and `game/hole.py`. The collision issue was in the way the game checked hitboxes and updated the score.

Step 2: Correct mole and hole behavior. We reduced the hitbox dimensions and adjusted the hole update and whack logic so a mole is deactivated after a successful whack or when its timer expires.

Step 3: Add game-over handling. The final score is displayed on a game-over screen when the round timer reaches zero, rather than relying only on terminal output.

Step 4: Add replay and difficulty selection. The game-over screen offers Easy, Medium, Hard, and Exit options. Choosing a difficulty starts a new round with its own spawn chance and mole duration. Keyboard shortcuts 1, 2, 3, and 4 are supported, with Q or Escape to exit.

Step 5: Add sound effects. Generated tones are used for successful whacks, missed clicks, and the end of a round. Playback depends on successful audio mixer initialization and available system audio.

Step 6: Update the main loop. The existing `main.py` entry point delegates event handling, updates, and rendering to the game engine and closes Pygame when the exit state is selected.

Difficulty configuration:

Mode	Spawn chance	Mole duration
Easy	0.012	65 frames
Medium	0.020	45 frames
Hard	0.032	30 frames

3. Code Validation and GitHub Submission

Step 7: Syntax check. We prepared the following command to check the Python files for syntax errors before running the game:

```
python -m py_compile main.py game\game_engine.py game\hole.py
```

Step 8: Run and review. The game was launched using `python main.py` for interactive review of gameplay, scoring, timer, game-over controls, and sound behavior.

Step 9: Prepare evidence. The before and after recordings were saved under `Lab-4/before_fix.mp4` and `Lab-4/after_fix.mp4`.

Step 10: Clean up and submit. A `.gitignore` was added for Python cache and virtual environment files, tracked bytecode cache files were removed, and the changes were committed as `Complete Whack-a-Mole Lab 4` and pushed to the personal GitHub repository.

4. Testing and Results

The following table lists the behavior to check in the running game; an item should only be marked as passed after it has been observed.

Test	Expected result
Whack an active mole	Score increases by exactly one.
Click empty space	Score stays unchanged.
Click near neighboring holes	One click awards no more than one point.
Mole timer expires	The active mole disappears.
Round timer reaches zero	Game-over screen shows the final score.
Choose a difficulty	A new round starts with the selected mode.
Miss click and round end	Corresponding tones play if audio is available.
Choose Exit	The game closes.

5. Conclusion

AI assistance guided the work from identifying the original double-scoring bug through updating the game logic, adding the requested features, preparing validation steps, and committing the project to GitHub. The final submission includes the updated source files, before/after video evidence, and this focused report.